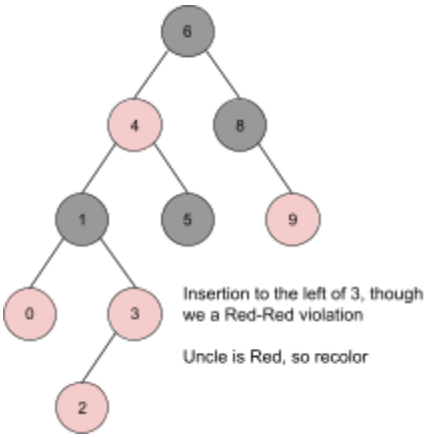
Indiana Atwood
CS 324
Homework #7
18 Mar 2026

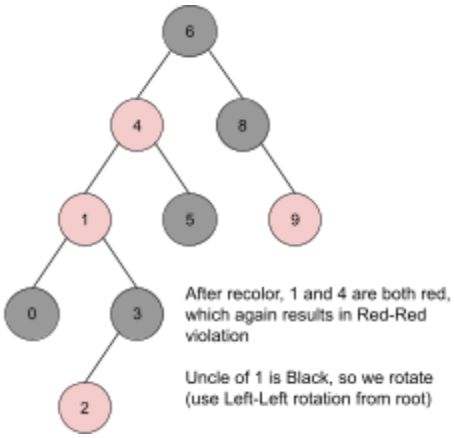
PROBLEM 1

Insert a 2 into the following red-black tree, using the transformations discussed in class.



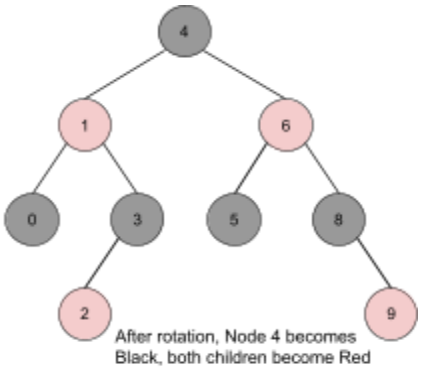
Insertion to the left of 3, though we a Red-Red violation

Uncle is Red, so recolor



After recolor, 1 and 4 are both red, which again results in Red-Red violation

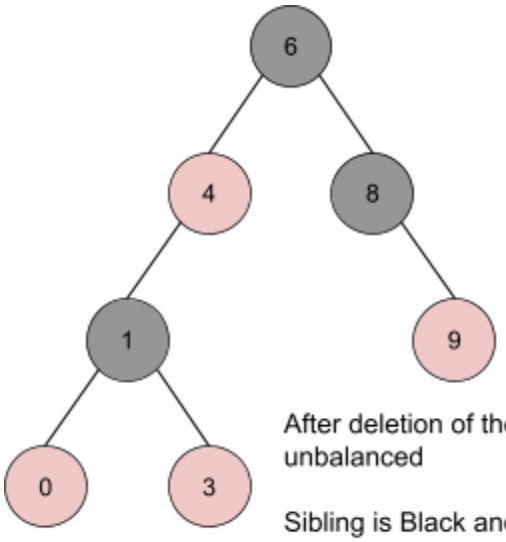
Uncle of 1 is Black, so we rotate (use Left-Left rotation from root)



After rotation, Node 4 becomes Black, both children become Red

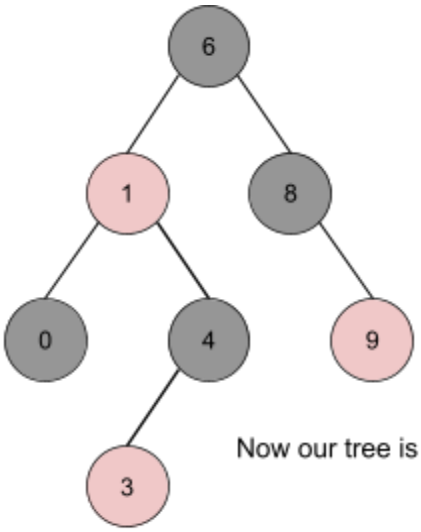
PROBLEM 2

Delete the 5 from the red-black tree depicted in the previous question; again, use the transformations discussed in class.



After deletion of the 5, the Black count is unbalanced

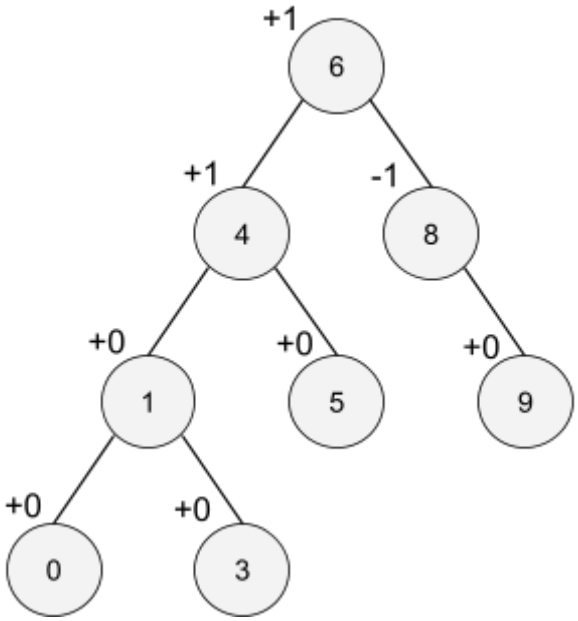
Sibling is Black and has at least one Red child, so rotate right at 4 and recolor



Now our tree is balanced

PROBLEM 3

Use the tree shown in Problem 1, except with all nodes colored white. Annotate it (with arrows) so that it is an AVL tree.

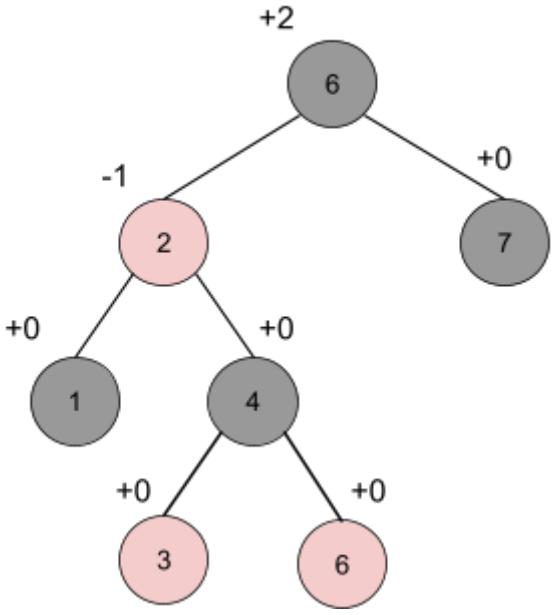


PROBLEM 4

Draw a red-black tree of 12 nodes or fewer that is not in the shape of a legal AVL tree. Explain why.

This is a valid Red-Black tree, where the root is black, there aren't two red nodes in a row, and every path has the same number of black nodes.

However, this is not a valid AVL tree, because the left hand-side has a max depth of 3 while the right is 1. The difference between them is +2, which violates an AVL tree.



PROBLEM 5

Find an optimal parenthesization of a matrix-chain product whose sequence of dimensions is $< 5, 10, 3, 12, 5, 50, 6 >$.

A: 5 x 10	B: 10 x 3	C: 3 x 12	D: 12 x 5	E: 5 x 50	F: 50 x 6

A x B: **150** (5 x 10 x 3)	B x C: **360** (10 x 3 x 12)	C x D: **180** (3 x 12 x 5)	D x E: **3000** (12 x 5 x 50)	E x F: **1500** (5 x 50 x 6)

A x (B x C): 960 (5 x 10 x 12) + 360 (A x B) x C: **330** (5 x 3 x 12) + 150	B x (C x D): **330** (10 x 3 x 5) + 180 (B x C) x D: 960 (10 x 12 x 5) + 360	C x (D x E): 4800 (3 x 12 x 50) + 3000 (C x D) x E: **930** (3 x 5 x 50) + 180	D x (E x F): **1860** (12 x 5 x 6) + 1500 (D x E) x F: 6600 (12 x 50 x 6) + 3000

A x (B x C x D): 580 (5 x 10 x 5) + 330 (A x B) x (C x D): **405** (5 x 3 x 5) + 150 + 180 (A x B x C) x D: 630 (5 x 12 x 5) + 330	B x (C x D x E): **2430** (10 x 3 x 50) + 930 (B x C) x (D x E): 9360 (10 x 12 x 50) + 360 + 3000 (B x C x D) x E: 2830 (10 x 5 x 50) + 330	C x (D x E x F): 2076 (3 x 12 x 6) + 1860 (C x D) x (E x F): **1770** (3 x 5 x 6) + 180 + 1500 (C x D x E) x F: 1830 (3 x 50 x 6) + 930

A x (B x C x D x E): 4930 (5 x 10 x 50) + 2430 (A x B) x (C x D x E): 1830 (5 x 3 x 50) + 150 + 930 (A x B x C) x (D x E): 6330 (5 x 12 x 50) + 330 + 3000 (A x B x C x D) x E: **1655** (5 x 5 x 50) + 405	B x (C x D x E x F): **1950** (10 x 3 x 6) + 1770 (B x C) x (D x E x F): 2940 (10 x 12 x 6) + 360 + 1860 (B x C x D) x (E x F): 2130 (10 x 5 x 6) + 330 + 1500 (B x C x D x E) x F: 5430 (10 x 50 x 6) + 2430

A x (B x C x D x E x F): 2250 (5 x 10 x 6) + 1950 (A x B) x (C x D x E x F): **2010** (5 x 3 x 6) + 150 + 1770 (A x B x C) x (D x E x F): 2550 (5 x 12 x 6) + 330 + 1860 (A x B x C x D) x (E x F): 2055 (5 x 5 x 6) + 405 + 1500 (A x B x C x D x E) x F: 3155 (5 x 50 x 6) + 1655

The optimal parthenthesization of the sequence $< 5, 10, 3, 12, 5, 50, 6 >$ is **2010.**

PROBLEM 6

Determine an Longest Common Subsequence of $< 1, 0, 0, 1, 0, 1, 0, 1 >$ *and* $< 0, 1, 0, 1, 1, 0, 1, 1, 0 >$.

String S = $< 1, 0, 0, 1, 0, 1, 0, 1 >$
String T = $< 0, 1, 0, 1, 1, 0, 1, 1, 0 >$

Cost	Str. T	**0**	**1**	**0**	**1**	**1**	**0**	**1**	**1**	**0**
Str. S	0	**0**	0	0	0	0	0	0	0	0
1	0	0	**1**	1	1	1	1	1	1	1
0	0	1	1	**2**	**2**	**2**	2	2	2	2
0	0	1	1	2	2	2	**3**	3	3	3
1	0	1	2	2	3	3	3	**4**	4	4
0	0	1	2	3	3	3	4	**4**	4	5
1	0	1	2	3	4	4	4	5	**5**	5
0	0	1	2	3	4	4	5	5	5	**6**
1	0	1	2	3	4	5	5	6	6	**6**

Dir.	Str. T	**0**	**1**	**0**	**1**	**1**	**0**	**1**	**1**	**0**
Str. S	F	**F**	F	F	F	F	F	F	F	F
1	F	T	**D**	L	D	D	L	D	D	L
0	F	D	T	**D**	**L**	**L**	D	L	L	D
0	F	D	T	D	T	T	**D**	L	L	D
1	F	T	D	T	D	D	T	**D**	D	L
0	F	D	T	D	T	T	D	**T**	T	D
1	F	T	D	T	D	D	T	D	**D**	T
0	F	D	T	D	T	T	D	T	T	**D**
1	F	T	D	T	D	D	T	D	D	**T**

Longest Common Subsequence = $< 1, 0, 0, 1, 1, 0 >$ or $< 1, 0, 1, 0, 1, 0 >$ (if S and T swap)

PROBLEM 7

Give an $O(n^2)$ – time algorithm to find the longest monotonically increasing subsequence of a sequence of n numbers.

Illustrate your algorithm on the sequence: $< 8, 3, 7, 5, 9, 3, 4, 1, 9, 2, 6 >$.

```
LONGEST-MONOTONICALLY-INCREASING-SUB(n, originalArr) {
     int finalLength = 0;
     int finalArr[] = null;
     for(int i = 0; i < n; i++) {
         int longestSub = 1;
         int longestArr[] = [originalArr[i]];
         int previousVal = originalArr[i];

         for(int j = i + 1; j < n; j++) {
             if(originalArr[j] >= previousVal) {
                 longestSub++;
                 longestArr.append(originalArr[j]);
                 previousVal = originalArr[j];
             }
         }

         if(longestSub > finalLength) {
             finalLength = longestSub;
             finalArr = longestArr;
         }
     }
 }
```

After computation (each iteration of loop #1):

0	{8, 9, 9}
1	{3, 7, 9, 9}
2	{7, 9, 9}
3	{5, 9, 9}
4	{9, 9}
5	{3, 4, 9}

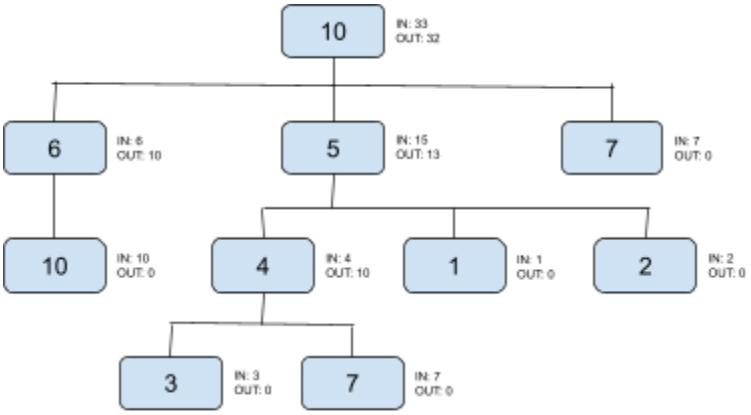
6	{4, 9}
7	{1, 9}
8	{9}
9	{2, 6}
10	{6}

With the original array of {8, 3, 7, 5, 9, 3, 4, 1, 9, 2, 6}, the longest monotonically increasing subsequence is {3, 7, 9, 9}.

PROBLEM 8

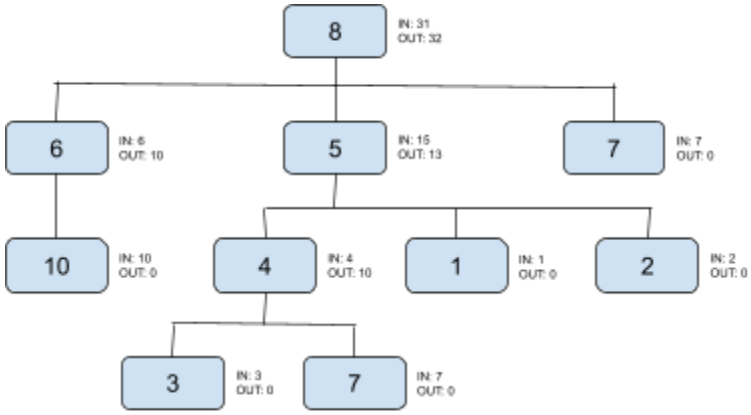
Describe an algorithm to make up a guest list that maximizes the sum of the conviviality ratings of guests. My code sets IN and OUT, I then follow the line of largest IN or OUT to find guests.

```
MAXIMIZE-CONVIVIALITY(u) {
    if (u == NULL) {
        return;
    }

    # Recursive call to children (post-order traversal)
    v = u.left_child;
    while (v != NULL) {
        MAXIMIZE-CONVIVIALITY(v);
        v = v.right_sibling;
    }

    # Calculate IN(u): u is invited, so children are not
    u.IN = u.rating;
    v = u.left_child;
    while (v != NULL) {
        u.IN += v.OUT;
        v = v.right_sibling;
    }

    # Calculate OUT(u): u is not invited, so max(IN, OUT)
    u.OUT = 0;
    v = u.left_child;
    while (v != NULL) {
        u.OUT += max(v.IN, v.OUT);
        v = v.right_sibling;
    }
}
```



President is IN, so 6, 5, 7 are OUT. Max conviviality: 33

Who can come: {10 (pres), 10 (staff), 3 (staff), 7 (staff), 2 (staff)}



President is OUT, so 6, 5, 7 can be IN. Max conviviality: 32

Who can come: {10 (staff), 5 (boss), 3 (staff), 7 (staff), 7 (boss)}

PROBLEM 9

Not just any greedy approach to the activity-selection problem produces a maximum-size set of mutually compatible activities. Give an example to show that the approach of selecting the activity of least duration from among those that are compatible with previously selected activities does not work.

PROBLEM 10

Give an efficient algorithm to find an optimal solution to this variant of the knapsack problem, and argue that your algorithm is correct.

```
KNAPSACK(n, items) {
     weight = 0;
     value = 0;
     optimal = {};

     for(int i = 0; i < n; i++) {
          if (weight + w[i] ≤ W) {     // W is weight limit
             optimal.add(i);
             weight += w[i];
             value += v[i];
          }
          else {
               break;
          }
     }

     return optimal;
}
```

This algorithm is able to find the most optimal version of the knapsack problem, as it puts the most valuable items in first. Because the items listed by increasing weight are of decreasing value, the lightest items are most important. This means we should opt to take the lighter items first (can put more in a knapsack *and* are most valuable). The algorithm is fairly simple with this version of the problem, where the elements listed by increasing weight are placed first.